M

Complete Reference Guide

Comparison · Logical · Element · Array · Update · Aggregation

TABLE OF CONTENTS

1	**Comparison Operators**	*$eq $ne $gt $gte $lt $lte $in $nin*
2	**Logical Operators**	*$and $or $not $nor*
3	**Element Operators**	*$exists $type*
4	**Array Operators**	*$all $elemMatch $size*
5	**Update Operators**	*$set $unset $inc $push $pull $addToSet*
6	**Aggregation Operators**	*$match $group $sort $project $limit*

1 · COMPARISON OPERATORS

COMPARISON OPERATORS

Filter documents based on field value comparisons.

$eq — Equal To

COMPARISON

Matches documents where the field value equals the specified value.

SYNTAX

```
{ field: { $eq: value } }
```

EXAMPLE

```
db.users.find({ age: { $eq: 25 } })
```

OUTPUT

```
{ "_id": 1, "name": "Alice", "age": 25 }
{ "_id": 4, "name": "Sam",   "age": 25 }
```

■ *Equivalent to { field: value } shorthand.*

$ne — Not Equal

COMPARISON

Matches all documents where the field value is not equal to the specified value.

SYNTAX

```
{ field: { $ne: value } }
```

EXAMPLE

```
db.users.find({ status: { $ne: "inactive" } })
```

OUTPUT

```
{ "_id": 1, "name": "Alice", "status": "active"  }
{ "_id": 2, "name": "Bob",   "status": "pending" }
```

■ *Also matches documents where the field does not exist.*

$gt / $gte — Greater Than

COMPARISON

$gt matches values greater than; $gte matches greater than or equal to.

SYNTAX

```
{ field: { $gt: value } }   // strictly greater
{ field: { $gte: value } }  // greater or equal
```

EXAMPLE

```
db.products.find({ price: { $gte: 100 } })
```

OUTPUT

```
{ "_id": 3, "product": "Laptop",  "price": 999 }
{ "_id": 5, "product": "Monitor", "price": 100 }
```

■ *Useful for numeric, date, and string range queries.*

$lt / $lte — Less Than

COMPARISON

$lt matches values less than; $lte matches less than or equal to.

SYNTAX

```
{ field: { $lt: value } }
{ field: { $lte: value } }
```

EXAMPLE

```
db.orders.find({ total: { $lt: 50 } })
```

OUTPUT

```
{ "_id": 2, "item": "Pen",    "total": 3.5  }
{ "_id": 7, "item": "Eraser", "total": 49.9 }
```

$in — Matches Any in Array

COMPARISON

Matches documents where the field value equals any value in the provided array.

SYNTAX

```
{ field: { $in: [value1, value2, ...] } }
```

EXAMPLE

```
db.users.find({ role: { $in: ["admin", "editor"] } })
```

OUTPUT

```
{ "_id": 1, "name": "Alice", "role": "admin"  }
{ "_id": 3, "name": "Carol", "role": "editor" }
```

■ *More efficient than multiple $or conditions.*

$nin — Matches None in Array

COMPARISON

Matches documents where the field value does NOT equal any value in the array.

SYNTAX

```
{ field: { $nin: [value1, value2, ...] } }
```

EXAMPLE

```
db.users.find({ status: { $nin: ["banned", "deleted"] } })
```

OUTPUT

```
{ "_id": 1, "name": "Alice", "status": "active"  }
{ "_id": 2, "name": "Bob",   "status": "pending" }
```

2 · LOGICAL OPERATORS

LOGICAL OPERATORS

Combine multiple query conditions using boolean logic.

$and — All Conditions Must Match

LOGICAL

Joins query clauses with a logical AND — returns documents that match ALL conditions.

SYNTAX

```
{ $and: [ { condition1 }, { condition2 }, ... ] }
```

EXAMPLE

```
db.products.find({
  $and: [ { price: { $gt: 50 } }, { inStock: true } ]
})
```

OUTPUT

```
{ "_id": 2, "name": "Chair",   "price": 120, "inStock": true }
{ "_id": 5, "name": "Lamp",    "price": 75,  "inStock": true }
```

■ *MongoDB applies $and implicitly when using a comma-separated conditions object.*

$or — At Least One Condition Matches

LOGICAL

Joins query clauses with a logical OR — returns documents that match ANY condition.

SYNTAX

```
{ $or: [ { condition1 }, { condition2 }, ... ] }
```

EXAMPLE

```
db.users.find({
  $or: [ { age: { $lt: 18 } }, { age: { $gt: 65 } } ]
})
```

OUTPUT

```
{ "_id": 1, "name": "Tom",  "age": 15 }
{ "_id": 6, "name": "Ruth", "age": 70 }
```

$not — Inverts Condition

LOGICAL

Inverts the effect of a query expression. Returns documents that do NOT match.

SYNTAX

```
{ field: { $not: { operator-expression } } }
```

EXAMPLE

```
db.inventory.find({ qty: { $not: { $gt: 100 } } })
```

OUTPUT

```
{ "_id": 1, "item": "paper", "qty": 50  }
{ "_id": 3, "item": "pens",  "qty": 100 }
```

■ *$not also matches documents where the field does not exist.*

$nor — None Must Match

LOGICAL

Performs a logical NOR — returns documents that fail ALL the query expressions.

SYNTAX

```
{ $nor: [ { condition1 }, { condition2 }, ... ] }
```

EXAMPLE

```
db.users.find({
  $nor: [ { status: "inactive" }, { age: { $lt: 18 } } ]
})
```

OUTPUT

```
{ "_id": 2, "name": "Alice", "status": "active", "age": 25 }
```

3 · ELEMENT OPERATORS

ELEMENT OPERATORS

Query documents based on field existence or BSON type.

$exists — Field Exists Check

ELEMENT

Matches documents that have (or do not have) the specified field.

SYNTAX

```
{ field: { $exists: <boolean> } }
```

EXAMPLE

```
db.users.find({ phone: { $exists: true } })
```

OUTPUT

```
{ "_id": 2, "name": "Bob",   "phone": "555-1234" }
{ "_id": 5, "name": "Eve",   "phone": "555-9876" }
```

■ *Use $exists: false to find documents missing a field.*

$type — BSON Type Check

ELEMENT

Matches documents where the field is of the specified BSON data type.

SYNTAX

```
{ field: { $type: <BSON type> } }
{ field: { $type: [type1, type2] } }  // multiple types
```

EXAMPLE

```
db.data.find({ value: { $type: "string" } })
```

OUTPUT

```
{ "_id": 1, "value": "hello"  }
{ "_id": 3, "value": "world"  }
```

■ *Common types: "string", "int", "double", "bool", "date", "array", "object", "null".*

4 · ARRAY OPERATORS

ARRAY OPERATORS

Query documents with array fields using powerful array matching.

$all — Array Contains All Values

ARRAY

Matches arrays that contain ALL of the specified elements (order doesn't matter).

SYNTAX

```
{ field: { $all: [value1, value2, ...] } }
```

EXAMPLE

```
db.courses.find({ tags: { $all: ["mongodb", "nosql"] } })
```

OUTPUT

```
{ "_id": 1, "title": "MongoDB Basics",    "tags": ["mongodb","nosql","db"] }
{ "_id": 3, "title": "NoSQL Deep Dive",   "tags": ["nosql","mongodb"] }
```

$elemMatch — Element-Level Matching

ARRAY

Matches documents where at least one array element satisfies ALL criteria.

SYNTAX

```
{ field: { $elemMatch: { condition1, condition2 } } }
```

EXAMPLE

```
db.students.find({
  scores: { $elemMatch: { subject: "math", score: { $gte: 90 } } }
})
```

OUTPUT

```
{ "_id": 1, "name": "Alice",
  "scores": [{"subject":"math","score":95},{"subject":"english","score":78}] }
```

■ *Without $elemMatch, conditions may match across different array elements.*

$size — Array Length Check

ARRAY

Matches documents where the array field has exactly the specified number of elements.

SYNTAX

```
{ field: { $size: number } }
```

EXAMPLE

```
db.posts.find({ comments: { $size: 3 } })
```

OUTPUT

```
{ "_id": 2, "title": "Hello World",
  "comments": ["Great!", "Nice post.", "Thanks!"] }
```

■ *$size does not accept ranges — use $where or $expr for that.*

5 · UPDATE OPERATORS

UPDATE OPERATORS

Modify documents in a collection with precision.

$set — Set Field Value

UPDATE

Sets the value of a field. Creates the field if it doesn't exist.

SYNTAX

```
db.collection.updateOne( { filter }, { $set: { field: value } } )
```

EXAMPLE

```
db.users.updateOne(
  { name: "Alice" },
  { $set: { age: 26, city: "Paris" } }
)
```

OUTPUT

```
{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }
// Document: { name: "Alice", age: 26, city: "Paris" }
```

$unset — Remove a Field

UPDATE

Removes the specified field(s) from a document entirely.

SYNTAX

```
db.collection.updateOne( { filter }, { $unset: { field: "" } } )
```

EXAMPLE

```
db.users.updateOne(
  { name: "Bob" },
  { $unset: { tempToken: "" } }
)
```

OUTPUT

```
// "tempToken" field is completely removed from the document
```

$inc — Increment / Decrement

UPDATE

Increments a field by the specified amount. Use negative values to decrement.

SYNTAX

```
db.collection.updateOne( { filter }, { $inc: { field: amount } } )
```

EXAMPLE

```
db.products.updateOne(
  { _id: 1 },
  { $inc: { stock: -1, soldCount: 1 } }
)
```

OUTPUT

```
// stock was 50  → now 49
// soldCount was 200 → now 201
```

■ *Creates the field with the increment value if it doesn't exist.*

$push — Add Element to Array

UPDATE

Appends a value to an array field. Creates the array if it doesn't exist.

SYNTAX

```
db.collection.updateOne( { filter }, { $push: { array: value } } )
```

EXAMPLE

```
db.posts.updateOne(
  { _id: 5 },
  { $push: { tags: "featured" } }
)
```

OUTPUT

```
// tags was ["news","tech"] → now ["news","tech","featured"]
```

■ *Use with $each to push multiple values: { $push: { tags: { $each: [...] } } }.*

$pull — Remove Element from Array

UPDATE

Removes all instances of a value (or matching condition) from an array.

SYNTAX

```
db.collection.updateOne( { filter }, { $pull: { array: value } } )
```

EXAMPLE

```
db.posts.updateOne(
  { _id: 5 },
  { $pull: { tags: "featured" } }
)
```

OUTPUT

```
// tags was ["news","tech","featured"] → now ["news","tech"]
```

$addToSet — Add Unique Element

UPDATE

Adds a value to an array only if it does not already exist (prevents duplicates).

SYNTAX

```
db.collection.updateOne( { filter }, { $addToSet: { array: value } } )
```

EXAMPLE

```
db.users.updateOne(
  { _id: 1 },
  { $addToSet: { hobbies: "reading" } }
)
```

OUTPUT

```
// If "reading" already in hobbies — no change.
// Otherwise it is appended to the array.
```

6 · AGGREGATION PIPELINE

AGGREGATION PIPELINE STAGES

Process and transform documents through a pipeline of sequential stages.

$match — Filter Stage

PIPELINE

Filters documents based on a condition — same syntax as find() queries.

SYNTAX

```
{ $match: { condition } }
```

EXAMPLE

```
db.orders.aggregate([
  { $match: { status: "shipped", total: { $gt: 100 } } }
])
```

OUTPUT

```
{ "_id": 3, "status": "shipped", "total": 250 }
{ "_id": 7, "status": "shipped", "total": 180 }
```

■ *Place $match early in the pipeline to improve performance.*

$group — Aggregate by Key

PIPELINE

Groups documents by a key and applies accumulator expressions.

SYNTAX

```
{ $group: { _id: <expr>, field: { <accumulator>: <expr> } } }
```

EXAMPLE

```
db.orders.aggregate([
  { $group: {
      _id: "$category",
      totalRevenue: { $sum: "$total" },
      count:        { $sum: 1 }
  } }
])
```

OUTPUT

```
{ "_id": "Electronics", "totalRevenue": 15000, "count": 42 }
{ "_id": "Clothing",    "totalRevenue": 8200,  "count": 30 }
```

■ *Common accumulators: $sum, $avg, $min, $max, $count, $push, $first, $last.*

$sort — Order Results

PIPELINE

Sorts documents. 1 = ascending, -1 = descending.

SYNTAX

```
{ $sort: { field: 1 | -1 } }
```

EXAMPLE

```
db.products.aggregate([
  { $sort: { price: -1 } }
])
```

OUTPUT

```
{ "name": "Laptop",  "price": 1299 }
{ "name": "Monitor", "price":  499 }
{ "name": "Mouse",   "price":   35 }
```

$project — Shape the Output

PIPELINE

Includes, excludes, or transforms fields in the output documents.

SYNTAX

```
{ $project: { field: 1, other: 0, newField: <expression> } }
```

EXAMPLE

```
db.users.aggregate([
  { $project: {
      fullName: { $concat: ["$firstName", " ", "$lastName"] },
      age: 1, _id: 0
  } }
])
```

OUTPUT

```
{ "fullName": "Alice Smith", "age": 25 }
{ "fullName": "Bob Jones",   "age": 30 }
```

■ *Use 0 to exclude fields and 1 to include them (cannot mix, except for _id).*

$limit — Cap Results

PIPELINE

Limits the number of documents passed to the next stage.

SYNTAX

```
{ $limit: <positive integer> }
```

EXAMPLE

```
db.logs.aggregate([
  { $sort:  { createdAt: -1 } },
  { $limit: 5 }
])
```

OUTPUT

```
// Returns only the 5 most recent log documents
```

■ *Combine with $sort and $skip to implement pagination.*

QUICK REFERENCE CHEAT SHEET

Operator	Category	Purpose
$eq / $ne	Comparison	Equal / Not equal
$gt / $gte	Comparison	Greater than (or equal)
$lt / $lte	Comparison	Less than (or equal)
$in / $nin	Comparison	In / Not in array
$and	Logical	All conditions must match
$or	Logical	Any condition matches
$not	Logical	Invert condition

$nor	Logical	None must match
$exists	Element	Field existence check
$type	Element	BSON type check
$all	Array	Array contains all values
$elemMatch	Array	Element-level array match
$size	Array	Array length equals N
$set	Update	Set field value
$unset	Update	Remove field
$inc	Update	Increment / Decrement
$push / $pull	Update	Add / Remove array element
$addToSet	Update	Add unique array element
$match	Pipeline	Filter stage
$group	Pipeline	Aggregate by key
$sort	Pipeline	Order results
$project	Pipeline	Shape output fields
$limit	Pipeline	Cap result count